

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра суперкомпьютеров и общей информатики

Отчет по лабораторной работе №2

Дисциплина: «Инженерия данных»

Выполнил: Вечканова П.А.

Группа: 6233-010402D

Самара 2025

ЗАДАНИЕ

Цели

1. Освоить оркестровку в p8n: триггеры, ветвления, бинарные данные, обработка ошибок.
2. Интегрировать внешние инструменты: ffmpeg, открытые STT-модели (Whisper и аналоги), LLM для перевода.
3. Реализовать полный цикл: Telegram Bot → загрузка/скачивание видео → извлечение аудио → распознавание речи → перевод EN→RU → формирование субтитров → инкрустирование в видео → отправка ответа в бота.

Постановка задачи (пайплайн)

1. Telegram Bot принимает либо ссылку на видео, либо сам файл видео.
2. Если пришла ссылка — скачать видео; если пришёл файл — использовать его напрямую. В обоих случаях извлечь аудиодорожку с помощью ffmpeg.
3. Сгенерировать субтитры (EN), используя auto_subtitle.
4. Выполнить перевод EN→RU с помощью LLM из HuggingFace (допустим API; предпочтительно — собственный сервер vLLM/LLama).
5. Добавить полученные (RU) субтитры в исходное видео.
6. Отправить результат обратно пользователю в Telegram.

Допускается работать только с английской речью на входе; многоязычие — как бонус.

ВЫПОЛНЕНИЕ РАБОТЫ

1. АРХИТЕКТУРА

Пайплайн реализован в n8n с использованием следующих технологий:

- n8n (версия 1.40+) — для оркестрации задач, ветвлений, обработки бинарных данных и повторных запусков.
- FFmpeg (6.1) — для извлечения аудио и наложения субтитров.
- Whisper API / auto_subtitle — для генерации субтитров (EN).
- Telegram Bot API — для взаимодействия с пользователем.

Выбор обусловлен высокой степенью интеграции, возможностью обработки бинарных данных и поддержкой внешних команд (Execute Command в n8n).

2. ИСТОЧНИКИ ДАННЫХ И РЕСУРСЫ

В качестве источника используются:

- Видео, переданное пользователем через Telegram-бота.

Также при написании кода использовались ресурсы:

- API Mistral – для перевода с английского языка. Было выбрано за бесплатные API key.
- Документация n8n – для настройки workflow
https://docs.n8n.io/?utm_source=n8n_app&utm_medium=app_sidebar
- Документация ngrok <https://ngrok.com/docs/what-is-ngrok>

А также не вошло в реализацию, но было испробовано:

- HuggingFace – для выбора локальной модели, но, к сожалению, модель была взята по API в связи с недостаточностью ресурсов и небольшой производительностью локальной модели

3. ОПИСАНИЕ РАБОТЫ

При получении сообщения от пользователя n8n определяет, является ли оно файлом или ссылкой.

- Если файл — сохраняется как бинарный объект.
- Если ссылка — скачивается видео.

Аудио извлекается из видео с помощью FFmpeg. Сгенерированное аудио передаётся в Whisper API для получения субтитров (EN). Субтитры отправляются на LLM для перевода на русский язык (RU). Переведённые субтитры сохраняются в формате .srt. Переведённые субтитры накладываются на исходное видео с помощью FFmpeg. Результат отправляется пользователю через Telegram Bot API. Временные файлы удаляются с помощью cleanup команды. На рисунке 1 приведен весь описанный выше путь в workflow n8n.

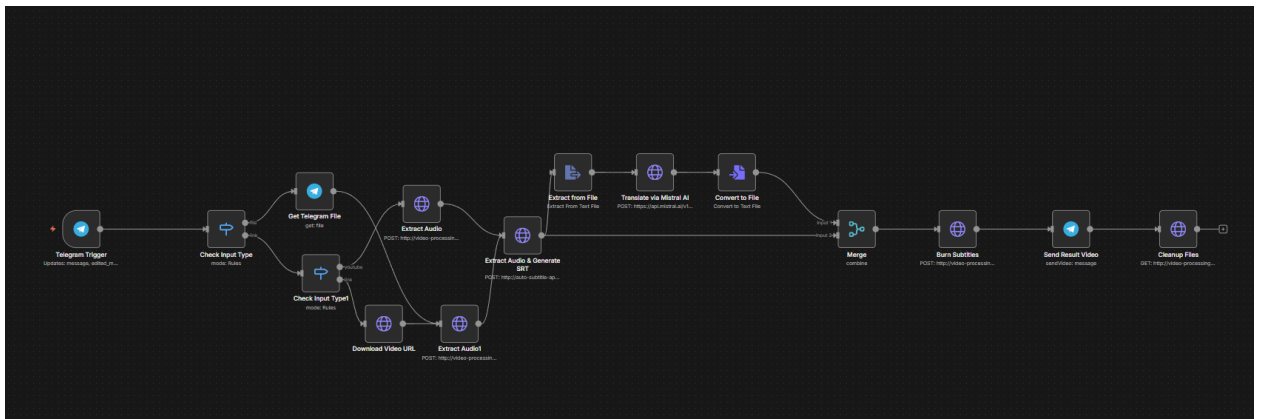


Рисунок 1 – Workflow

4. КАЧЕСТВО ДАННЫХ И ОБРАБОТКА ОШИБОК

Реализована обработка ошибок:

- Проверка HTTP-статусов при обращении к API.
- Повторные попытки при ошибках (retry).
- Уведомление пользователя о сбое через Telegram.
- Автоматическая очистка временных файлов после завершения или сбоя.

5. ВЫПОЛНЕНИЕ ПАЙПЛАЙНА И РЕЗУЛЬТАТЫ

Первым этапом работы была настройка телеграм бота с помощью @BotFather и получение ID чата для дальнейшей настройки работы.

Далее, написав код и настроив docker-compose, запускаем контейнер с помощью команды `docker-compose up -d` (пример запуска и вывод представлены на рисунке 2)

```

PS C:\Users\vechk\etl_weather\de-lab-2-2025> docker compose up -d
time="2025-12-11T14:57:48+04:00" level=warning msg="project has been loaded without an explicit name from a symlink. Using name \"de-lab-2-2025\"
"
[+] Running 3/3
 ✓ Container video-processing Running 0.0s
 ✓ Container whisper-api Running 0.0s
 ✓ Container n8n-lab2 Started 5.3s
PS C:\Users\vechk\etl_weather\de-lab-2-2025>
  
```

Рисунок 2 – Запуск контейнера

Запускаем ВПН и далее запускаем ngrok. С помощью ngrok мы будем настраивать вебхуки для телеграм. Регистрируемся в ngrok и забираем токен. Добавляем токен к себе в конфиг (рисунок 3).

```

C:\WINDOWS\system32>ngrok config add-authtoken 36f5ba4H5VXJUAp3sS9KqKYVqJh_4oTHBefWhdagdfb8TXFNX
Authtoken saved to configuration file: C:\Users\vechk\AppData\Local\ngrok\ngrok.yml
  
```

Рисунок 3 – Добавление токена ngrok

Далее запускаем прослушивание нашего порта ngrok `http 5678` и видим окно с рисунка 4.

```
C:\Program Files\WindowsApps\ngrok\ngrok_3.24.0.0_x64_1g87z0zv29zzc\ngrok.exe - ngrok http 5678

ngrok

Free Users: Agents s3.18.x stop connecting 12/17/25. Update or upgrade: https://ngrok.com/pricing

Session Status
Account      online
Account     vechkanova.p@gmail.com (Plan: Free)
Update      update available (version 3.34.0, Ctrl-U to update)
Version      3.24.0-msix
Region      Europe (eu)
Latency      65ms
Web Interface http://127.0.0.1:4040
Forwarding   https://heartfelt-listlessly-claude.ngrok-free.dev -> http://localhost:5678

Connections
      ttl    opn    rt1    rt5    p50    p90
      4      0     0.00   0.00   6.10   6.24

HTTP Requests
-----
14:30:43.105 +30 POST /webhook-test/61161219-d350-47e9-be41-e772948295c6/webhook 200 OK
14:24:29.215 +24 POST /webhook-test/61161219-d350-47e9-be41-e772948295c6/webhook 200 OK
14:16:03.974 +16 POST /webhook-test/61161219-d350-47e9-be41-e772948295c6/webhook 200 OK
14:14:42.104 +14 POST /webhook-test/61161219-d350-47e9-be41-e772948295c6/webhook 200 OK
14:14:42.232 +14 POST /webhook-test/61161219-d350-47e9-be41-e772948295c6/webhook 200 OK
```

Рисунок 4 – Запущенный ngrok

Проверим все существующие эндпоинты через интерфейс Fast API (рисунок 5)/

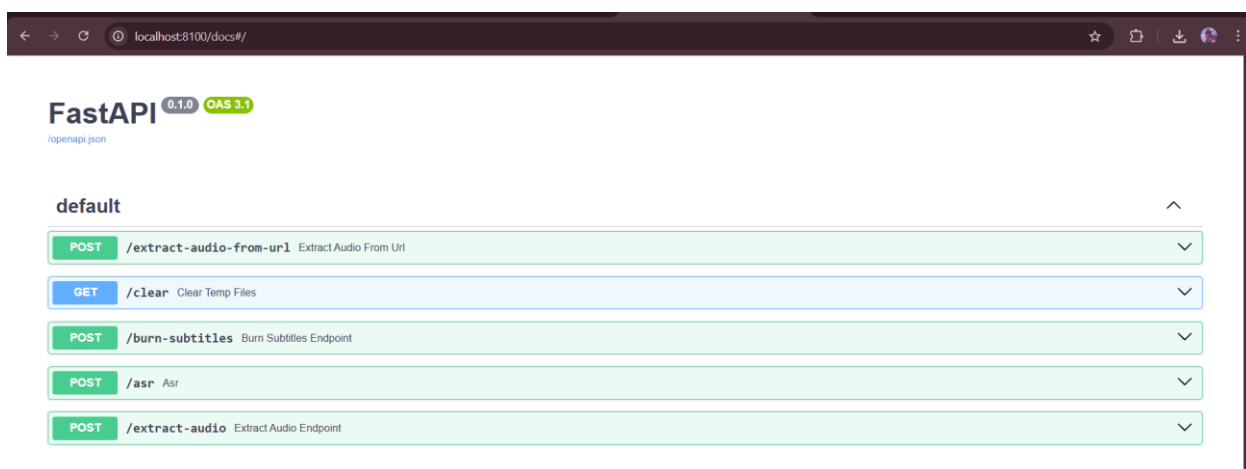


Рисунок 5 – Созданные пост и гет запросы

Заходим в n8n для настройки workflow. Настроенный workflow представлен на рисунке 6.

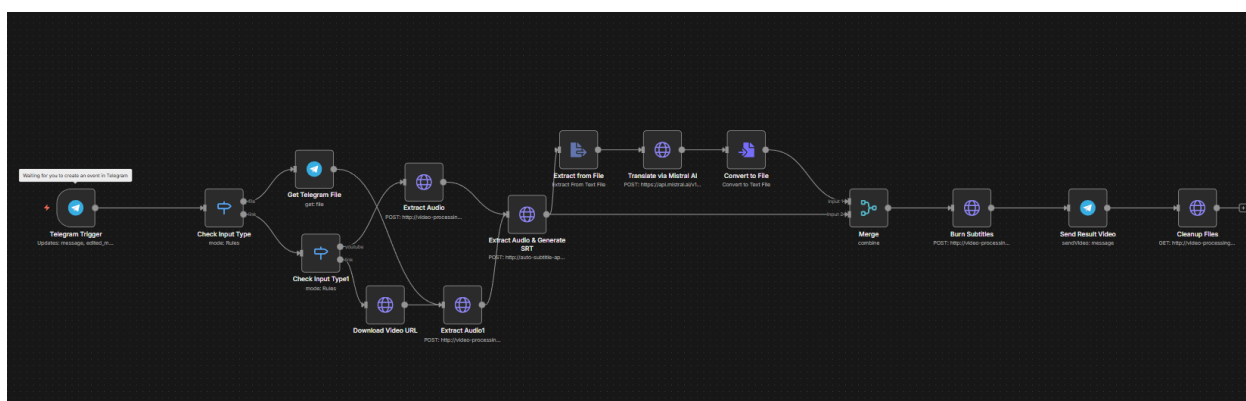


Рисунок 6 – Workflow

Проверяем работу бота, отправив ссылку на видео и само видео. Пример успешной работы на рисунках 7-10.

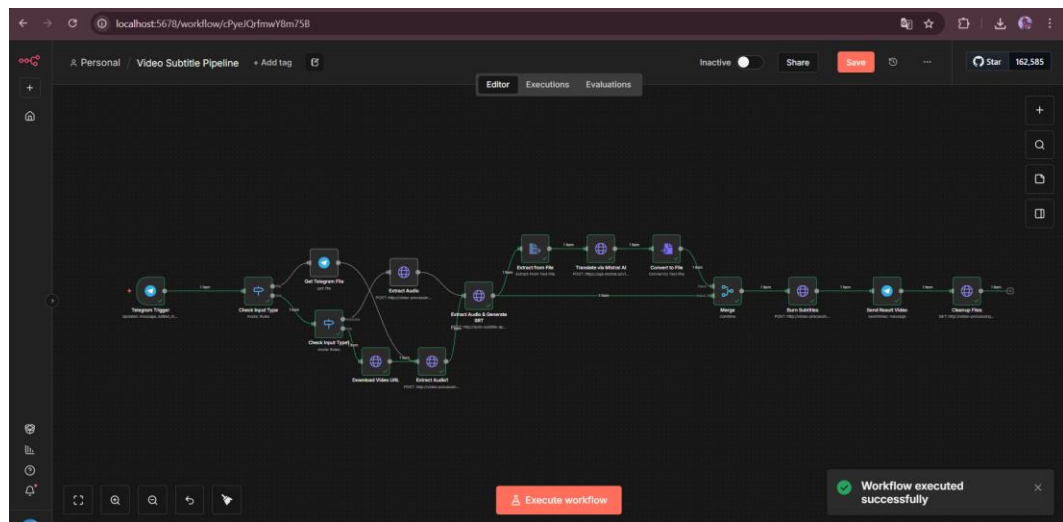


Рисунок 7 – Успешно выполненный воркфлоу

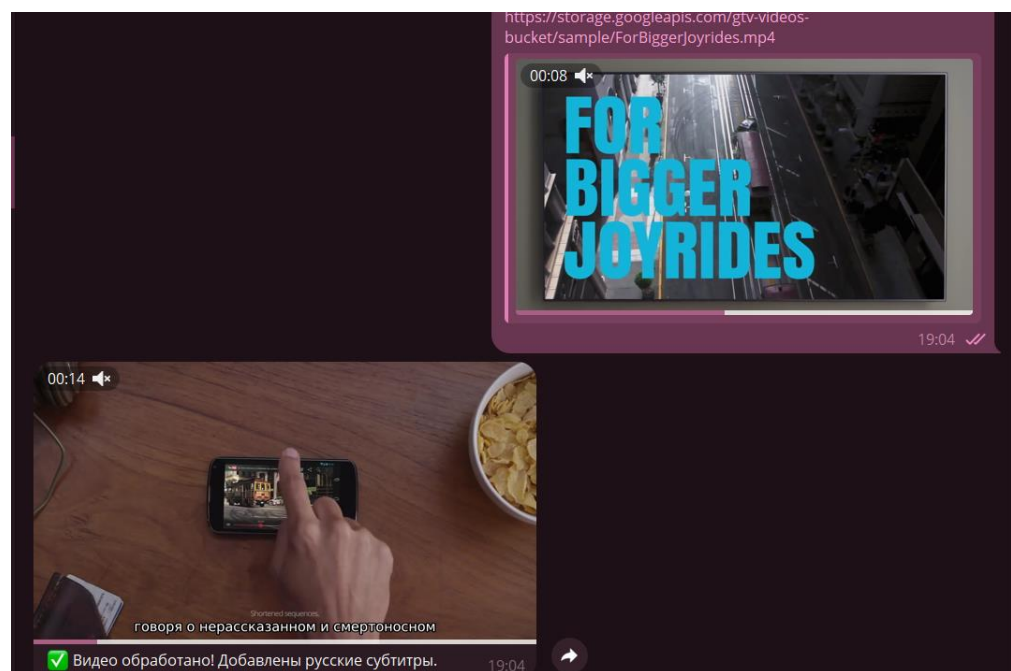


Рисунок 8 – Пример ответа на ссылку

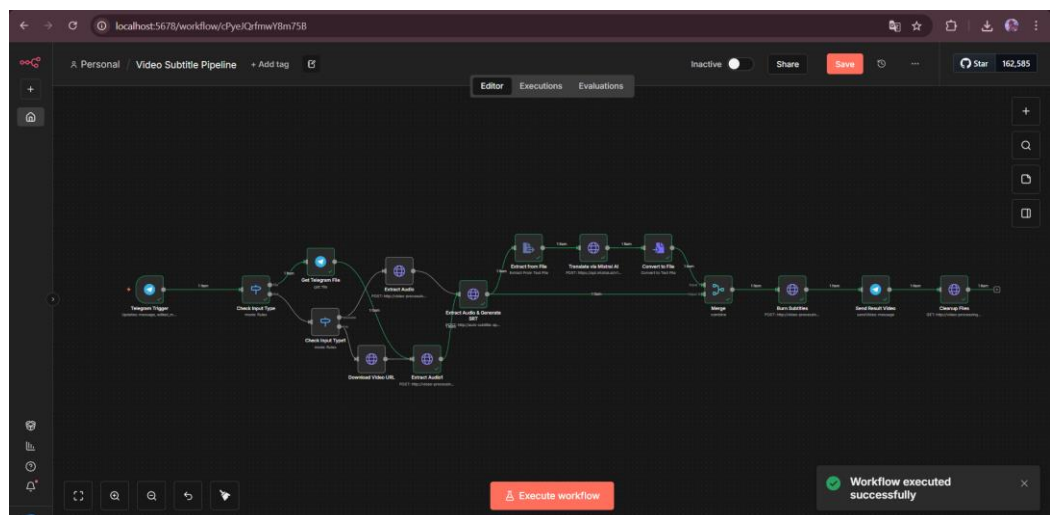


Рисунок 9 – Успешно выполненный воркфлоу

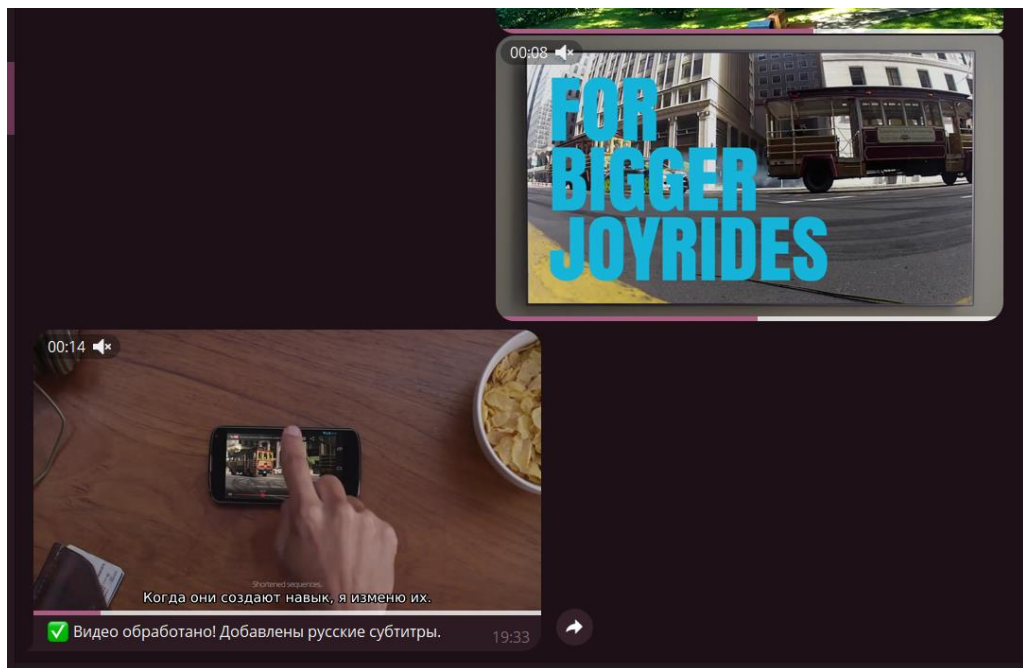


Рисунок 10 – Пример ответа на видео

Также посмотрим логи в докере. Видно, что отправка и очистка произошли успешно

```
auto-subtitle-api | /app/.venv/lib/python3.10/site-packages/whisper/transcribe.py:132: UserWarning: FP16 is not supported on CPU; using FP32 instead
                    warnings.warn("FP16 is not supported on CPU; using FP32 instead")
                    INFO:      172.18.0.3:46860 - "POST /asr HTTP/1.1" 200 OK

video-processing | INFO:      172.18.0.3:57700 - "POST /burn-subtitles HTTP/1.1" 200 OK
                  INFO:      172.18.0.3:49146 - "GET /clear HTTP/1.1" 200 OK
```

Рисунок 9 – Логи в докере при успешном завершении

ВЫВОДЫ

В ходе выполнения лабораторной работы выполнены все поставленные цели: реализован полный цикл обработки видео — от получения через Telegram-бота до отправки пользователю с наложенными русскими субтитрами. Собран и отлажен workflow в n8n, настроена интеграция с Whisper API и LLM для генерации и перевода субтитров, а также использован FFmpeg для извлечения аудио и наложения субтитров. Настроено окружение с Docker, включая обработку ошибок, очистку временных файлов и отправку уведомлений через Telegram-бота. По мере работы также были протестированы различные LLM для перевода.

Возникали трудности с настройкой окружения, в частности, у меня через раз запускался контейнер с n8n, а также с настройкой телеграмм-триггера, поэтому пришлось использовать сторонние инструменты, например ngrok.